

Agent Harness Awareness: Turning Verified Telemetry into Operational Control for Tool-Using Agents

Manu Agrawal · Shrey Nagpal

Abstract

Autonomous agents increasingly act through tools, yet their harnesses keep the operational facts that shape an action's cost and likelihood of success—latency, failures, route health, deadlines, quotas, and resource pressure—outside the agent's decision loop. This creates **operational blindness**: semantically equivalent actions can have radically different operational consequences while appearing identical to the model. We introduce **Agent Harness Awareness**, a trust-separated control architecture that converts bounded, fresh, host-verified telemetry into an agent decision input while keeping measurement, enforcement, and evaluation outside the model. We evaluate its tools-and-network instance in two separately analysed, hash-audited studies. In the primary 216-decision direct-provider ToolRoute ablation across GPT-5.6 Sol, Claude Sonnet 5, and Gemini 3.7 Flash, verified tool-health state reduced mean observable policy regret by **96.9%** against a neutral same-shape interface (4,252 ms to 131 ms) and raised completion from **69.4% to 95.8%**. In a separate 27-decision end-to-end replication using live local TCP/HTTP execution, OpenTelemetry client spans, and Toxiproxy-injected faults, verified state reduced mean regret from **8,956 ms to 0 ms** and completed the live action in **9/9** model-by-fault cases, versus **5/9** for the neutral control. Together, these studies demonstrate a practical systems principle: telemetry should not end at human dashboards. Selected, provenance-preserving operational state can serve as a high-leverage control input for tool-using agents.

1. From operational blindness to closed-loop control

An agent can understand a user request and every tool's semantics yet still make an avoidable operational error. The missing information is often not semantic; it is the current state of the harness: which route is slow, failing, rate-limited, stale, resource-constrained, or no longer economical. When the harness knows this state but the agent does not, equivalent APIs remain indistinguishable at the moment the agent selects one.

This is a general agent-runtime problem, not merely a routing problem. Harnesses already observe four high-value operational dimensions: **tools and network** (route health and circuit state); **runtime** (deadlines, queueing, and concurrency); **hardware and sandbox** (memory headroom, CPU pressure, storage); and **economics** (context headroom, token burn, quota, and cost). A harness-aware agent can use these facts for alternate-route selection, deadline-aware decomposition, checkpointing before budget exhaustion, resource-aware concurrency, and backpressure-aware coordination.

We call this gap **operational blindness**. Agent Harness Awareness closes it by reducing bounded, freshness-labelled, host-verified execution state into an agent decision interface. In the proposed interface, the host retains authority over collection, reduction, safety, and enforcement. The model may adapt its next strategic action, but cannot rewrite measurements, disable constraints, or modify the evaluator.

This paper contributes a general systems paradigm, a trust-separated state interface, a frozen 216-decision multi-model information ablation, and a separately analysed 27-decision OpenTelemetry/Toxiproxy transport replication. The contribution is not a new tool-call syntax or model ranking. It is evidence that what an agent knows about its harness can be as consequential as what it knows about the user task. This addresses a practical reliability gap: controlled tool/API failures and dynamic replanning remain difficult even when ordinary task execution succeeds [8, 9].

2. Agent Harness Awareness

At decision time t , let x_t denote the task and tool interface, h_t the host state, and $z_t = \varphi(h_t)$ a bounded host-generated projection. The proposed **Substrate State Telemetry (SST) contract** is a minimal, trust-separated interface covering Tools (success history, latency, error/circuit state), Runtime (deadlines and exit state), Hardware (memory, CPU, disk, processes), and Economics (context headroom, quota, rate, cost). ToolRoute evaluates the Tools instance: each route row contains a probe-window count, successes, latency EWMA, consecutive-failure/error state, circuit state, and observation age; the snapshot header carries its freshness bound. The other dimensions are direct future instantiations, not pooled evidence. Agent-Native Telemetry develops compact, verifiable telemetry representation and transport; Harness Awareness asks the complementary behavioral question: whether selected verified state changes the consequential operational action an agent chooses [7].

Harness Awareness complements rather than replaces deterministic systems controls. The data plane—kernel, proxy, circuit breaker, retry policy, and scheduler—continues to own immediate recovery and safety; routine replica failover belongs there. The agent control plane acts when current state changes a semantic or strategic choice: select an alternate provider or data source, defer work, adopt a degraded-mode plan, or allocate time and budget differently.

3. Two complementary ToolRoute studies

ToolRoute evaluates the tools-and-network dimension through two distinct studies. Study 1 is the primary controlled information ablation; Study 2 is an independently reported systems replication. Their data are never pooled.

Study	Question	Evidence	Role
Study 1	Does truthful state improve route selection when the interface is held constant?	216 frozen provider decisions; seeded monitor	Primary behavioral estimate
Study 2	Does the effect survive an OTel-instrumented live HTTP tool path?	27 frozen decisions; OTel, Toxiproxy, live action	End-to-end replication

3.1 Study 1: controlled information ablation

The agent retrieves a read-only customer record through one of two functionally equivalent routes, `tool_alpha` and `tool_beta`, or chooses `wait`. ToolRoute is a minimal benchmark: it holds task semantics constant while varying current operational state across semantically substitutable actions. This isolates operational decision quality from task-solving difficulty, tool-schema complexity, and hidden middleware behavior. The host owns monitoring, schedules, execution, and an external observable-cost oracle. Each decision is an independent full-checkpoint episode; models have no shell, filesystem, network-tool, schedule, oracle, or artifact access.

Condition	Model-visible information
A: task only	Task and equivalent tool descriptions; no operational projection.
B: neutral envelope	Same telemetry-shaped envelope, fields, row order, and tool rows as C, with equal neutral values.
C: verified state	Same envelope with fresh host-owned facts for current route state.

A is the deployment baseline: task and tool semantics alone, approximating conventional agent behavior without harness state. **B is the structural control:** the same telemetry-shaped interface with all route-relevant values neutralized. In Study 1, each route row is `window=6 succ=6 consec_fail=0 latency_ewma=1000.0ms last_err=NONE circuit=CLOSED age=10ms`; both routes receive the same values. In Study 2, the identical row structure uses `window=3` with the same shared neutral values. **C is the intervention:** verified current state. Thus A-to-C measures practical deployment gain, while B-to-C supports attributing the difference to route-relevant verified information rather than interface shape or formatting. B is not an exact provider-token-matching control; input-token counts are descriptive.

We report two complementary outcomes. **Observable policy regret** is the avoidable observable operational cost of the selected action:

$$r_{\text{obs}}(a, z) = c_{\text{obs}}(a, z) - \min_{a'} c_{\text{obs}}(a', z)$$

For a route with monitor window n , s successes, latency EWMA l , and circuit state q , let $p_f = (n - s)/n$ when $n > 0$, and $p_f = 0.5$ when $n = 0$. The frozen observable cost is $c_{\text{obs}} = l + 10,000p_f + 10,000 \cdot 1[q = \text{OPEN}]$ ms. Choosing wait incurs $500 + 10,000 - \min c_{\text{obs}}$ ms: it defers for 500 ms and misses the required record, so it is neither free nor a hidden retry. The host computes this only from canonical monitor facts rendered in C. **0 ms regret means that the agent selected an observable-best action with no excess operational cost, not that the request itself took zero time.** The evaluator never uses hidden latent anomalies unavailable to the model. **Completion** is a separate binary outcome: whether the selected route successfully retrieves the requested record; in Study 2, whether the post-decision live HTTP action succeeds. A route can complete yet still have positive regret when it was avoidably slower or riskier.

The frozen direct-provider cohort contains 216 independent decisions (three model families $\times$ six seeds $\times$ four predeclared environment states per seed $\times$ three conditions). A state denotes a distinct full-checkpoint decision episode, not a sequential conversation turn; every attempt retains its prompt, sanitized request/response, snapshot, tool event, evaluation result, and finalization hashes. No request was retried.

3.2 Study 2: live OpenTelemetry transport replication

The separately labelled v0.2 replication preserves the A/B/C information intervention but moves to a live instrumented tool path: instrumented HTTP client to Toxiproxy to equivalent local HTTP route; OpenTelemetry spans to a host-owned reducer to bounded agent state to model decision to live selected-route execution. Each episode makes three monitor requests per route through local TCP/HTTP proxies. The deterministic reducer converts the six spans into per-route window counts, successes, latency EWMA ($\alpha = 0.2$), error/circuit state, and a 2 s freshness bound. Toxiproxy injects a predeclared 300 ms latency, connection, or HTTP 503 failure on one route; the HTTP client timeout is 2 s, and the selected route then executes through the same still-live proxy. The frozen replication contains 27 independent decisions (three models $\times$ three fault regimes $\times$ A/B/C). Provider/model, condition, faulted route, option order, execution order, no-retry policy, and manifest and binary hashes were fixed before execution; raw spans, reducer events, provider records, action spans, results, and finalization hashes are retained.

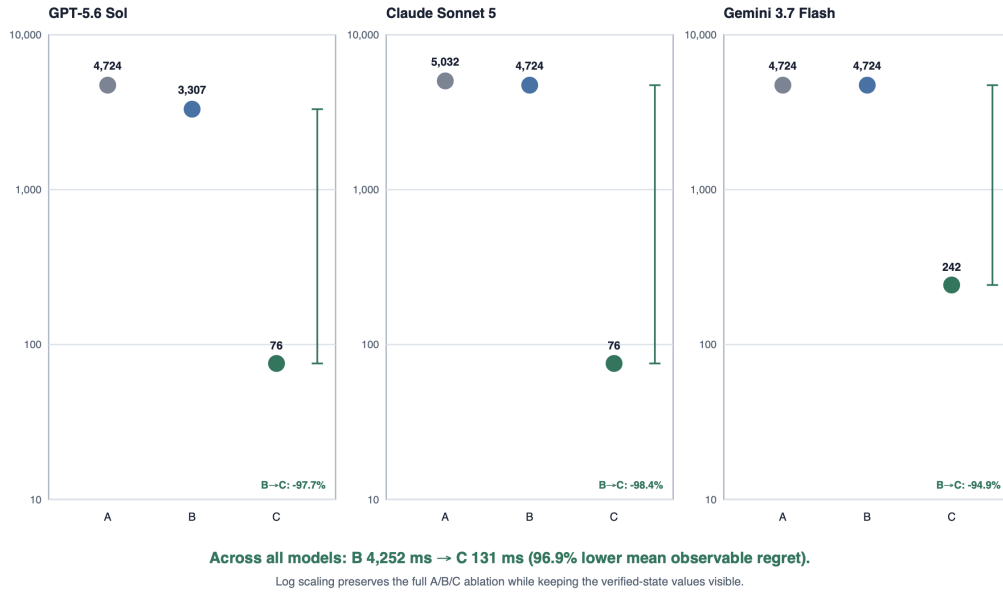
4. Results

4.1 Study 1: controlled information ablation

Against the task-only deployment baseline A, verified state C reduces mean observable policy regret from 4,826.45 ms to 131.12 ms: a **97.3% reduction**. Completion rises from 47/72 (65.3%; Wilson 95% CI 53.8-75.2%) to 69/72 (95.8%; Wilson 95% CI 88.5-98.6%), a **30.6-point increase**. The structural B-to-C comparison reaches the same conclusion while holding interface shape constant: regret falls from 4,251.54 ms to 131.12 ms (**96.9% lower**) and completion rises from 50/72 (69.4%; Wilson 95% CI 58.0-78.9%) to 69/72 (95.8%; Wilson 95% CI 88.5-98.6%), a **26.4-point increase**. A descriptive bootstrap over the 18 model-by-seed blocks gives a B-to-C regret difference of 4,120 ms (95% interval 2,543-5,893 ms) and a completion gain of 26.4 points (18.1-34.7 points). Together, these comparisons show both practical deployment value and the informational contribution of verified telemetry.

Figure 1. Verified state collapses observable regret across all information conditions

Condition means on a logarithmic scale; n = 24 independent direct-provider decisions per point. Lower is better.



Frozen ToolRoute v1.0 cohort. Condition means are independent samples; points are not paired trajectories.

Figure 1. Mean observable policy regret for all three information conditions in the frozen 216-decision cohort. Log scaling keeps verified-state values visible; A, B, and C are independent condition means, not trajectories.

Condition	Role	Mean regret	Completion
A	Task-only deployment baseline	4,826.45 ms	47/72 (65.3%)
B	Neutral structural control	4,251.54 ms	50/72 (69.4%)
C	Verified host state	131.12 ms	69/72 (95.8%)

The result is present in every model family. GPT-5.6 Sol, Claude Sonnet 5, and Gemini 3.7 Flash respectively achieve C completion of 23/24, 23/24, and 23/24; their C mean regrets are 75.50 ms, 75.50 ms, and 242.35 ms. The three residual C outcomes remain in the denominator: each selected wait, which misses the required record under the frozen action contract.

4.2 Study 2: live transport replication

The transport replication reproduces the central decision pattern through a live, OpenTelemetry-instrumented HTTP tool path. The task-only deployment baseline A completed 6/9 live actions (66.7%; Wilson 95% CI 35.4-87.9%) with 6,734.30 ms mean observable policy regret. The neutral structural control B completed 5/9 actions (55.6%; Wilson 95% CI 26.7-81.1%) with 8,956.37 ms regret. Verified state C selected the observable-best route in **9/9 cases**, completed **9/9 live actions** (Wilson 95% CI 70.1-100.0%), and incurred **0.00 ms** regret. Thus C improves both the practical task-only baseline and the same-shape structural control, while the B-to-C comparison supports the information effect under the stated structural control.

Model	A succ.	A regret	B succ.	B regret	C succ.	C regret
GPT-5.6 Sol	2/3	6,767.64	1/3	13,434.67	3/3	0.00
Claude Sonnet 5	2/3	6,668.05	2/3	6,667.18	3/3	0.00
Gemini 3.7 Flash	2/3	6,767.22	2/3	6,767.25	3/3	0.00

Model	A succ.	A regret	B succ.	B regret	C succ.	C regret
Total	6/9	6,734.30	5/9	8,956.37	9/9	0.00

Figure 2. Verified telemetry improves every live transport decision

Frozen OpenTelemetry/Toxiproxy v0.2 replication: 27 independent remote-model decisions; selected route executed live after each decision.



Figure 2. Study 2 live transport replication. The upper panel shows that all nine verified-state C cases selected the observable-best route and completed the subsequent live proxied HTTP action. The lower panel reports mean observable policy regret and live completion for all A/B/C conditions, highlighting the B-to-C structural-control comparison. Here, 0 ms regret denotes zero excess observable operational cost, not zero HTTP duration.

The predeclared schedule varies both the faulted route and option order across model-by-regime cells; the exact 27-cell allocation is retained in the frozen manifest. Across A and B, models selected tool_alpha in 17 of 18 decisions, including 7 of 8 cases in which tool_beta was listed first. This is an observed default route-label preference in this cohort, not a claim about universal positional bias. Condition C removed its operational consequence by selecting the healthy route in every regime regardless of which route was degraded or listed first.

5. Implications, boundaries, and agenda

Modern observability systems collect traces, metrics, and logs for human operators. Harness Awareness supplies the missing control-plane step: select the facts relevant to the immediate decision, preserve provenance and freshness, and expose the bounded result to the agent without giving it authority over truth or enforcement. The same interface pattern can support alternate-region routing, deadline-aware planning, quota-aware work decomposition, context-budget checkpointing, resource-aware concurrency, and multi-agent backpressure. The correct interface is not a raw telemetry dump; it is bounded, redacted, freshness-labelled, and host-owned. Scaling it to large tool catalogs and persistent trajectories motivates compact state deltas, selective/top-k projection, and explicit context-serialization budgets.

The evidence is strong for the mechanism tested here: current verified tool-health facts improve selection between functionally equivalent routes. The primary cohort uses a synthetic host-owned monitor; the replication uses a minimal local HTTP service and deterministic proxy faults. The paper establishes the Tools instance of the broader SST contract; it does not estimate uncontrolled production-outage rates, universal service topologies, or the effects of stale state between observation and action. Those are the next high-value tests, alongside multi-turn reactive recovery, not prerequisites for the demonstrated pre-dispatch control-loop effect.

6. Conclusion

Agent Harness Awareness closes an operational loop that tool-using systems often leave open: the harness observes execution state while the agent chooses as if that state did not exist. In the frozen controlled cohort, verified state substantially outperformed both the task-only deployment baseline and the neutral same-shape structural control; the latter supports attributing the improvement to verified operational information rather than interface formatting. In the separately reported OpenTelemetry/Toxiproxy replication, that same information intervention selected the observable-best route and completed every C action across three providers and three fault regimes. The evidence supports a clear systems direction: turn selected, verified telemetry into an active agent control input while keeping truth and enforcement in the harness.

7. Artifact availability

The complete evaluation archive is available at <https://github.com/manu2/scac-dynamic-agent-harness> [10]. It preserves frozen manifests, complete retained study directories, sanitized provider records, model-visible prompts, host observations, raw OpenTelemetry spans, reducer outputs, action results, finalization hashes, derived analyses, figures, and PDF-build scripts. The two study cohorts remain separately labelled and are never pooled.

Appendix A. Reproducibility record

The Study 1 task template asks the model to retrieve the next required record through functionally equivalent tool_alpha and tool_beta routes, return exactly one listed action, and choose from a predeclared rotation of the two routes plus wait. Condition A receives that task only. Condition B receives the same fixed telemetry envelope and tool-row structure as C, with identical neutral values for both routes. Condition C receives the host-rendered full checkpoint. The archive records the exact prompt and option order for every included decision.

Study 1 and Study 2 use gpt-5.6-sol, claude-sonnet-5, and gemini-3.7-flash, a 1,024-token output limit, no provider-native tools, and no request retries. OpenAI and Anthropic requests omit an explicit temperature field; Gemini requests set temperature to 0.0. For Study 2, the frozen manifest fixes the model, condition, faulted route, option order, and execution order for all 27 episodes. Each monitor phase performs three OpenTelemetry-instrumented requests per route; the selected route then executes over the still-live local TCP/HTTP proxy.

The repository analysis scripts validate the frozen 216-cell Study 1 grid and the 27 finalised Study 2 episodes before emitting derived results. Their outputs are checked against retained finalization hashes; they make no provider calls and do not rewrite experiment artifacts [10].

References

- [1] Shunyu Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023. arXiv:2210.03629.
- [2] Timo Schick et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS, 2023. arXiv:2302.04761.
- [3] Xiao Liu et al. AgentBench: Evaluating LLMs as Agents. ICLR, 2024. arXiv:2308.03688.
- [4] Shunyu Yao et al. tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. ICLR, 2025. arXiv:2406.12045.
- [5] OpenTelemetry. Specification. <https://opentelemetry.io/docs/specs/> (accessed 2026-09-04).
- [6] Shopify. Toxiproxy. <https://github.com/Shopify/toxiproxy> (accessed 2026-09-04).
- [7] Jun He and Deying Yu. Agent-Native Telemetry: Verifiable State-Delta Evidence for Autonomous Operations. 2026. arXiv:2608.16178.
- [8] Aayush Gupta. ReliabilityBench: Evaluating LLM Agent Reliability Under Production-Like Stress Conditions. 2026. arXiv:2601.06112.
- [9] Dongsheng Zhu et al. When Tools Fail: Benchmarking Dynamic Replanning and Anomaly Recovery in LLM Agents. 2026. arXiv:2606.05806.
- [10] Manu Agrawal and Shrey Nagpal. SCAC Dynamic Agent Harness: ToolRoute Evaluation Artifacts. <https://github.com/manu2/scac-dynamic-agent-harness> (accessed 2026-09-04).